

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 11 · LECTURE NOTES

Visibility and Motion Planning

What can be seen, and how to get there.

MODULE OUTCOMES

- Compute a visibility polygon by angular sweep.
- Build a visibility graph and use it for shortest paths.
- Explain configuration space and the Minkowski sum.
- Explain how a navigation mesh is built and why.
- Compare roadmap, sampling, and grid-based planners.

1 Visibility polygons

VISIBILITY FROM A POINT IS A ROTATIONAL SWEEP

Sort every obstacle vertex by its angle about the viewpoint, and sweep a ray through them in that order. This is the plane sweep of Module 04 with an angular rather than a linear parameter.

Maintain the set of obstacle edges the ray currently crosses, ordered by distance from the viewpoint — the status structure. Edges enter when the ray reaches their first endpoint and leave at their second.

The nearest edge in the status structure is what is visible in the current direction. Whenever the nearest edge changes — because a closer edge was inserted, or the current nearest was removed — emit a vertex of the visibility polygon.

$O(n \log n)$, dominated by the angular sort. This is the foundation of two-dimensional field-of-view cones, line-of-sight checks, guard coverage (Module 05 §4), and 2D dynamic lighting and shadow casting.

THE DEGENERACIES ARE THE USUAL SUSPECTS

Collinear vertices — two obstacle corners at exactly the same angle from the viewpoint. The sweep order between them is undefined, and choosing wrongly inserts an edge before removing the one it should replace, corrupting the nearest-edge determination.

The viewpoint lying exactly on an obstacle edge or at a vertex. This is extremely common in practice, because agents stand against walls and in doorways — it is the normal case, not an edge case, which is Module 01 §4 in miniature.

Angle comparison invites atan2, which is transcendental and immediately destroys any exactness established in Module 02. **Compare angles with orient instead**, after bucketing by half-plane — the same correction as Module 03 §1.

And the sweep wraps around at $\pm\pi$, so the comparator must be cyclic rather than linear. **A classic source of subtle bugs**: the visibility polygon comes out correct except for one wrong vertex near the start angle, which is hard to spot and easy to misattribute.

2 Configuration space

SHRINK THE ROBOT TO A POINT, GROW THE OBSTACLES

Planning a collision-free motion for a shape is hard; planning for a point is easy. The difficulty is entirely in the robot having extent, because then collision depends on the robot's whole footprint at every position along the path.

So transform the problem. Replace each obstacle O with the Minkowski sum $O \oplus (-R)$, where R is the robot shape taken about its reference point. **The robot becomes a single point** moving among the enlarged obstacles.

A point path through the grown obstacles corresponds exactly to a collision-free motion of the original shape — the correspondence is exact, not approximate, which is what makes the transformation worth doing.

For a translating convex robot the sum is convex and computable in $O(n + m)$, by merging the two polygons' edge vectors in angular order. That is remarkably cheap. **Rotation adds a dimension** — the configuration space becomes three-dimensional for a planar robot — **and the obstacles in it are no longer polygonal**, which is where the problem becomes genuinely hard.

$$A \oplus B = \{ a + b : a \in A, b \in B \}$$

Convex $\oplus$ convex is $O(n + m)$ by merging edge vectors in angular order, and the result is convex with at most $n + m$ edges. **Non-convex $\oplus$ non-convex is $O(n^2 m^2)$ in the worst case** and the result may have holes. **This gap is why convex decomposition (Module 03 §3) is everywhere in collision and planning** — it is the same reason GJK requires convexity (Module 12), and the same reason navmesh cells are convex (§3).

3 Roadmaps and navigation meshes

Method	Idea	Character
Visibility graph	Connect every pair of mutually visible obstacle vertices, then run a shortest-path search on the resulting graph.	Produces the genuinely shortest path in the plane, because an optimal path bends only at obstacle vertices. The graph is $O(n^2)$, which limits it to modest scenes.
Navigation mesh	Cover the walkable surface with convex polygons, search the adjacency graph, then smooth.	What games actually use. Cell count tracks geometric complexity rather than area, and the funnel algorithm produces a natural path (below).
Grid with A*	Discretise the world uniformly and search.	Simple and robust, and paths look discretised — diagonal stair-stepping that needs post-smoothing. Cost scales with area, not complexity.
Probabilistic roadmap (PRM)	Sample random configurations, keep collision-free ones, connect nearby pairs.	Handles high-dimensional configuration spaces, and amortises well over many queries in a static world.
RRT and RRT*	Grow a tree from the start toward random samples.	Single-query and fast; plain RRT gives poor paths, while RRT* converges to optimal given time.

WHY A NAVMESH BEATS A GRID

A grid discretises space uniformly, so a large empty room costs exactly as many cells as an equally large cluttered one, and movement is constrained to the grid's directions — producing the characteristic stair-stepped path that needs a smoothing pass to look acceptable.

A navigation mesh covers the walkable surface with convex polygons, so a single polygon can be an entire room. The cell count tracks the complexity of the geometry rather than its area, which is a very large saving in open environments.

Convexity is the whole point. Within one convex cell, straight-line movement between any two points is guaranteed collision-free, so the path search only ever needs to make cell-to-cell decisions — the within-cell motion is free.

Then the funnel algorithm pulls the path taut through the sequence of shared edges (the 'portals') between consecutive cells, producing the shortest path within that corridor in linear time. The result looks natural rather than stepped, and it is optimal given the corridor — though not necessarily globally, since the corridor itself came from a graph search.

- 1 **Voxelise the world geometry**, marking each voxel solid or open.
- 2 **Filter by walkability**: apply a maximum slope, a maximum step height, the agent's required clearance height, and ledge detection. **This is where agent parameters enter**, which is why a navmesh is built per agent size.
- 3 **Extract the walkable surface** — the top faces of the remaining spans — and build a distance field giving each cell's distance to the nearest obstacle or ledge.

- 4 **Watershed-partition the distance field into regions**, so that region boundaries fall in corridors rather than across open space, then trace each region's contour.
- 5 **Simplify the contours, triangulate them** (Module 05), and merge adjacent triangles into larger convex polygons wherever convexity is preserved.
- 6 **And erode everything by the agent radius** — which is exactly the Minkowski sum of §2, applied as a distance-field threshold rather than computed polygonally. **Recast does precisely this**, and it is the standard open implementation worth reading.

4 Sampling-based planning

SAMPLING GIVES UP COMPLETENESS, DELIBERATELY

A robot arm with seven revolute joints has a **seven-dimensional configuration space**, and the obstacles in it are the high-dimensional images of the physical obstacles under the forward kinematics. **Constructing that space explicitly is hopeless** — Module 10 §3's volume arguments apply in full.

So sample it instead: draw random configurations, discard those in collision (which only requires a collision *test*, never an explicit obstacle representation), connect nearby surviving samples with local paths, and search the resulting graph.

This is probabilistically complete — the probability of finding a path approaches 1 as samples increase, if one exists. **It can never report that no path exists**, only fail to find one, which is a real limitation when the answer matters.

And it struggles with narrow passages. A thin corridor in configuration space has small volume and is therefore rarely sampled, so the planner may run for a very long time on a problem whose solution is geometrically obvious. **That is the characteristic failure mode**, and a large fraction of the sampling-based planning literature is about mitigating it — bridge sampling, obstacle-based sampling, and medial-axis sampling (Module 08 §4) all target exactly this.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
LaValle — Planning Algorithms (free book)	The reference for §2 and §4 , free in full. Configuration space, PRM, and RRT from the author of RRT.
de Berg et al. — Computational Geometry, chapters 13 and 15 (robot motion planning, visibility graphs)	Minkowski sums and visibility graphs, with the complexity analyses.

Resource	What it covers for this module
Mikko Mononen — Recast and Detour (free)	The navmesh pipeline of §3, implemented. The standard open-source navigation toolkit; the source is the documentation.
Amit Patel — Red Blob Games, pathfinding and visibility (free)	Interactive explanations of A*, the funnel algorithm, and 2D visibility. Unusually good for building intuition before reading the formal treatments.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a visibility polygon by angular sweep and render it.
- 2 **Place the viewpoint exactly on a vertex and on an edge** and fix what breaks.
- 3 Replace atan2 comparison with orient-based comparison and verify the output is unchanged.
- 4 Implement the Minkowski sum of two convex polygons in $O(n + m)$.
- 5 **Grow a set of obstacles by a disc robot** and verify that a point path through the grown obstacles is collision-free for the original.
- 6 Build a visibility graph and find shortest paths. Compare the path length against A* on a grid.
- 7 **Implement the funnel algorithm** over a sequence of navmesh portals and compare against the unsmoothed cell-centre path.
- 8 Generate a navmesh from a simple scene by the six-step pipeline.
- 9 Implement PRM and RRT for a 2D disc robot and compare path quality and time.
- 10 **Construct a narrow-passage problem** and measure how sample count scales with the passage width.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the visibility sweep and what its status structure holds.
- 2 Name four degeneracies in the visibility sweep.
- 3 What is configuration space and what transformation builds it?
- 4 State the Minkowski sum and give the convex and non-convex costs.
- 5 Compare five planning methods on optimality and cost.
- 6 Give three reasons a navmesh beats a grid.
- 7 Why must navmesh cells be convex?
- 8 What does the funnel algorithm do?

9 What is probabilistic completeness, and what is the narrow-passage problem?